

BudgetBench: A Budget-Constrained Evaluation Harness for Tool-Using Code Agents

Chenrui Xu,^{*} Hanyu Jin,^{*} and Sylvia Li^{*}

Carnegie Mellon University, Pittsburgh, PA, USA

E-mail: chenrui2@andrew.cmu.edu; hanyuj@andrew.cmu.edu; yangli6@andrew.cmu.edu

Abstract

Existing code-agent benchmarks rank agents by terminal correctness and ignore how many tool calls each agent spent. Two agents that solve the same issue, one in 5 calls and one in 50, score identically though their deployment costs differ by an order of magnitude. We introduce **BudgetBench**, an evaluation harness that makes the tool-call budget a first-class axis: it intercepts every action, debits its cost, terminates the episode when the ceiling is reached, and reports Success@B, AUC of the success-versus-budget curve, and budget violation rate with bootstrap confidence intervals. We instantiate the harness on a deterministic Knapsack environment and on SWE-bench Lite, and study three agents: a single-shot baseline, a ReAct-style loop, and a regime-switching BudgetAwareAgent that adapts to remaining budget. Across 450 Knapsack and 96 SWE-bench Lite episodes (Gemini 2.5 Flash and Pro on SWE-bench; heuristic agents on Knapsack), BudgetAware matches or beats the single-shot baseline with zero violations, while the loop agent collapses at low budgets. Upgrading from Gemini Flash to Pro adds nothing on SWE-bench at $B = 20$ but ~ 12 percentage points at $B = 40$ for multi-step agents, suggesting that under tight budgets agent design dominates model capability.

1 Introduction

Modern code-agent benchmarks ask a simple question: does the submitted patch pass the hidden test suite? SWE-bench¹ and earlier function-level benchmarks like HumanEval² and MBPP³ are silent on the cost of producing each solution. Yet practitioners care intensely about cost: an agent that fixes a bug in 5 tool calls dominates one that needs 50, even when both score 1.0 on standard leaderboards. The benchmark cannot see the difference; the engineering team cannot ignore it.

The gap distorts agent design. ReAct-style loops⁴ over-explore because each additional search, inspect, or test call costs them nothing in the metric. Downstream systems such as SWE-Agent⁵ inherit the same incentive. Pass@k² measures sample efficiency across attempts but not action efficiency *within* an attempt. HELM⁶ reports per-query token cost as a post-hoc audit rather than a per-episode constraint. The classical anytime literature⁷ studied quality-versus-deliberation trade-offs decades ago, but modern LLM benchmarks have not adopted these ideas.

This paper contributes (1) BudgetBench, a small harness that exposes a per-episode tool-call budget as a first-class axis; (2) two reference environments at very different scales (a deterministic 0/1-Knapsack toy and SWE-bench Lite); (3) three reference agents including a regime-switching BudgetAwareAgent that adapts to remaining budget; and (4) an empirical study spanning 546 episodes with a Gemini 2.5 Flash-versus-Pro comparison on SWE-bench. Our main finding is that, in the budget-constrained regime, a small amount of strategy beats a large amount of model: BudgetAware adds about 120 lines of code but produces larger gains than upgrading the LLM backbone.

2 Methods

2.1 Framework

BudgetBench is built around four abstractions. An *Agent* exposes `act(obs, budget_remaining, history)` and returns a tool call. An *Environment* accepts the call, returns an *Observation*, and provides a final scoring hook. A *BudgetRunner* glues them, intercepts every call, debits its cost, and terminates the episode when the running total exceeds the configured ceiling. Each run produces an immutable *Episode* trace with one of four statuses: `OK_SOLVED`, `TEST_FAIL`, `INVALID_FORMAT`, or `BUDGET_EXCEEDED` (Figure 1). The runner itself is roughly 200 lines of Python and has no LLM-specific dependencies; an existing tool-using agent typically needs under 50 lines of glue to opt in.

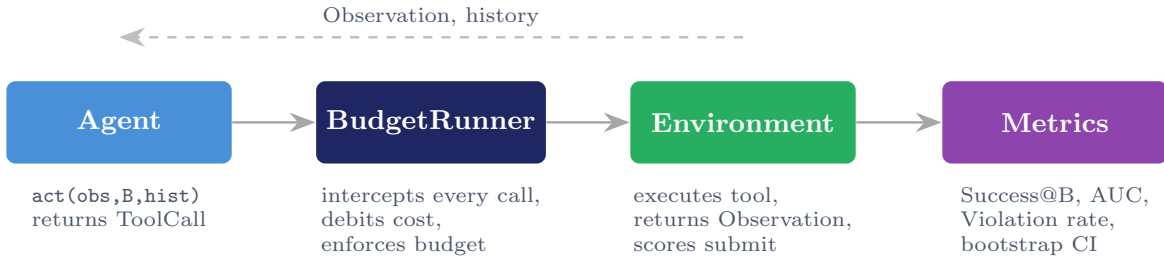


Figure 1: BudgetBench architecture. The BudgetRunner intercepts every tool call, debits its cost, and terminates the episode the moment the budget is exhausted.

Tool costs reflect real expense. Each lightweight call (search, inspect, edit, hierarchy, and the Knapsack manipulators) costs one unit. `run_tests` costs three, since real test suites take seconds to minutes. `submit_patch` costs zero, because once the agent submits, further work is wasted.

2.2 Metrics

For N episodes with statuses s_i and budget usage b_i ,

$$\begin{aligned}\text{Success@B} &= \frac{1}{N} \sum \mathbb{I}[s_i = \text{OK_SOLVED} \wedge b_i \leq B] \\ \text{Violation} &= \frac{1}{N} \sum \mathbb{I}[s_i = \text{BUDGET_EXCEEDED}] \\ \text{AUC} &= \frac{1}{B_{\max}} \int_0^{B_{\max}} \text{Success@B} dB\end{aligned}$$

We approximate the integral by the trapezoidal rule and accompany Success@B with a 95% bootstrap CI (1000 resamples). We also report TTFV, the mean budget consumed before the agent’s first valid `submit_patch`. AUC is our primary ranking metric: it rewards agents that succeed across the budget range, not only at $B_{\max}$.

2.3 Environments

Knapsack. Each episode generates ten items with weights and values uniform in $[1, 20]$ given a fixed seed; the agent selects a subset fitting capacity 50 with maximum value. A dynamic-programming oracle scores submissions. We use Knapsack purely as a harness shakedown: deterministic, no API key, isolates agent strategy from LLM stochasticity. The Knapsack agents in our reported experiments are heuristic, not LLM-backed.

SWE-bench Lite.⁸ Each episode runs a Docker container from the official evaluation image at the issue’s base commit, exposing six tools (`search`, `inspect`, `hierarchy`, `edit`, `run_tests`, `submit_patch`). Scoring follows a two-tier strategy: we first attempt the official harness, and fall through to a manual fallback (re-applying the patch in a fresh sandbox and running the issue’s test sets directly) when the harness reports a recoverable error. We use eight default-ordering Lite issues (four `astropy`, four `django`).

2.4 Agents

SingleShotAgent issues exactly one LLM call: read issue, emit unified diff, submit. Its budget consumption is independent of the ceiling, isolating raw model capability.

LoopAgent is a standard ReAct loop. At each step it sends issue + history + tool list to a Gemini chat session and parses one tool call, repeating until `submit_patch` or budget exhaustion. Crucially, the prompt does not include the remaining budget.

BudgetAwareAgent (key novelty) extends LoopAgent with three regimes by remaining-budget fraction f (Figure 2). *HIGH* ($f > 0.6$): full toolset, behaves like LoopAgent. *MEDIUM* ($0.3 < f \leq 0.6$): prunes `run_tests` from the action set, roughly tripling remaining exploratory moves. *LOW* ($f \leq 0.3$): skips the LLM call, extracts the most recent valid diff from history, and submits. We reuse LoopAgent through three hook overrides; the new code is roughly 120 lines. Thresholds are hand-tuned and identical across our experiments.

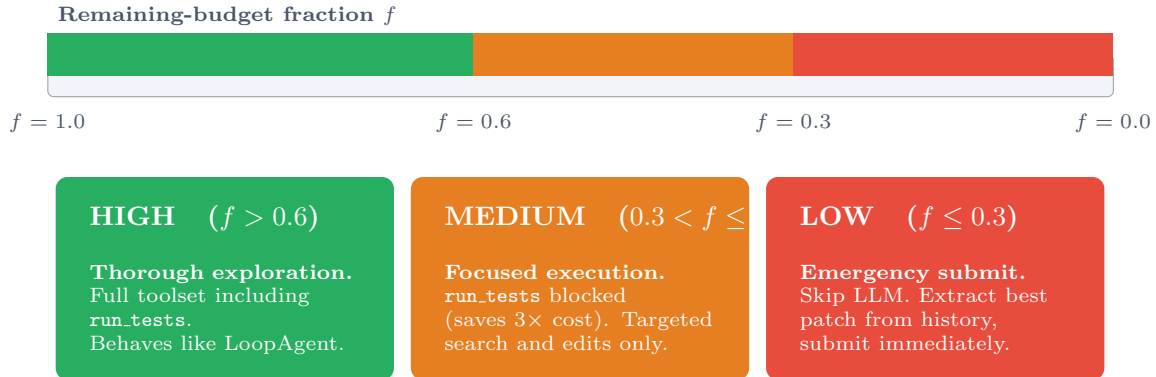


Figure 2: BudgetAwareAgent’s three regimes. HIGH explores freely; MEDIUM blocks the expensive `run_tests`; LOW skips the LLM and submits the best patch seen so far.

2.5 Experiment design

We launch experiments through a single `batch_experiment.py` entry point that fixes seeds per (environment, issue). Cells: Knapsack with the heuristic agents at $B \in \{5, 10, 20\}$ over 50 seeds (450 episodes; the Knapsack agents do not call an LLM and so a Flash-versus-Pro

split is not meaningful), and SWE-bench Lite with Flash and Pro at $B \in \{20, 40\}$ over 8 issues (48 episodes per model). Total: 546 episodes.

3 Results

3.1 Knapsack

SingleShot and BudgetAware reach $\text{Success@B} = 0.62$ at every budget we test (Table 1). LoopAgent collapses at low budgets: it scores zero at $B \in \{5, 10\}$ and 0.08 at $B = 20$, and all low-budget failures are `BUDGET_EXCEEDED`. BudgetAware’s violation rate is zero across its 150 Knapsack episodes; LoopAgent’s is 100% at $B \leq 10$.

Table 1: Knapsack: Success@B with 95% bootstrap CIs ($n=50$ seeds), AUC at $B_{\max} = 20$, and TTFV (mean budget before first valid submit). Agent abbreviations: Single = SingleShotAgent, Loop = LoopAgent, Budget = BudgetAwareAgent. The Knapsack agents are heuristic and do not call an LLM, so we do not split this table by model; see Limitations. BudgetAware ties SingleShot on Success@B but loses on AUC because it explores before falling back.

Agent	S@5	S@10	S@20	AUC	TTFV
Single	0.62	0.62	0.62	0.59	1.0
Loop	0.00	0.00	0.08	0.02	17.9
Budget	0.62	0.62	0.62	0.39	8.0

The TTFV column explains the AUC gap. SingleShot submits at $\text{TTFV} = 1$. BudgetAware’s TTFV is exactly $0.4 \cdot B$ (2, 4, 8 at $B = 5, 10, 20$), which matches the budget it consumes in the HIGH regime before the 0.6 threshold tips it into the fallback path; the relationship is mechanical, not emergent. LoopAgent only submits at $B = 20$, and consumes 17.9 of 20 units before doing so.

3.2 SWE-bench Lite

The qualitative picture changes (Table 2, Figure 3). SingleShot solves none of the 16 SWE-bench episodes per model: without exploration, the LLM cannot guess a patch that resolves

any of the eight selected astropy and django issues. BudgetAware achieves the highest Success@B in the table at $B = 20$ (0.625, five of eight issues) under both Flash and Pro, a 25-percentage-point lead over LoopAgent at the same budget; its violation rate stays at zero whereas LoopAgent’s Flash violation rate is 38%.

Table 2: SWE-bench Lite (eight issues per cell; agent abbreviations as in Table 1). The strongest cell is Budget at Pro $B=20$ (5/8 solved, 0% violation). Pro eliminates Loop’s Flash budget violations entirely.

Agent	Model	B	Succ@B	AUC	Viol%
Single	Flash	20	0.000	0.000	0%
Single	Flash	40	0.000	0.000	0%
Single	Pro	20	0.000	0.000	0%
Single	Pro	40	0.000	0.000	0%
Loop	Flash	20	0.375	0.091	38%
Loop	Flash	40	0.375	0.164	13%
Loop	Pro	20	0.375	0.259	0%
Loop	Pro	40	0.500	0.441	0%
Budget	Flash	20	0.625	0.322	0%
Budget	Flash	40	0.375	0.173	0%
Budget	Pro	20	0.625	0.378	0%
Budget	Pro	40	0.500	0.438	0%

The Flash-to-Pro lift on SWE-bench differs from Knapsack. At $B = 20$ Pro is no better than Flash for any agent; at $B = 40$ both LoopAgent and BudgetAware gain 12.5 pp. Pro’s stronger reasoning needs more steps to materialise. Pro also slashes LoopAgent’s violation rate to zero, which makes Pro both more accurate per call and more efficient per step. The best cell is BudgetAware Pro at $B = 20$: 62.5% success, zero violations, the best Success@B in the entire matrix.

3.3 Failure modes

Aggregating Flash episodes across both budgets, SingleShot fails uniformly (16/16 `TEST_FAIL`). LoopAgent splits between `TEST_FAIL` (37.5%) and `BUDGET_EXCEEDED` (25%). BudgetAware shows zero `BUDGET_EXCEEDED`; its failures are all `TEST_FAIL` (50%), the failure mode that is recoverable with a better patch rather than more budget. Splitting LoopAgent’s failures by budget confirms the expected shift: at $B = 20$ three of five failures are `BUDGET_EXCEEDED`,

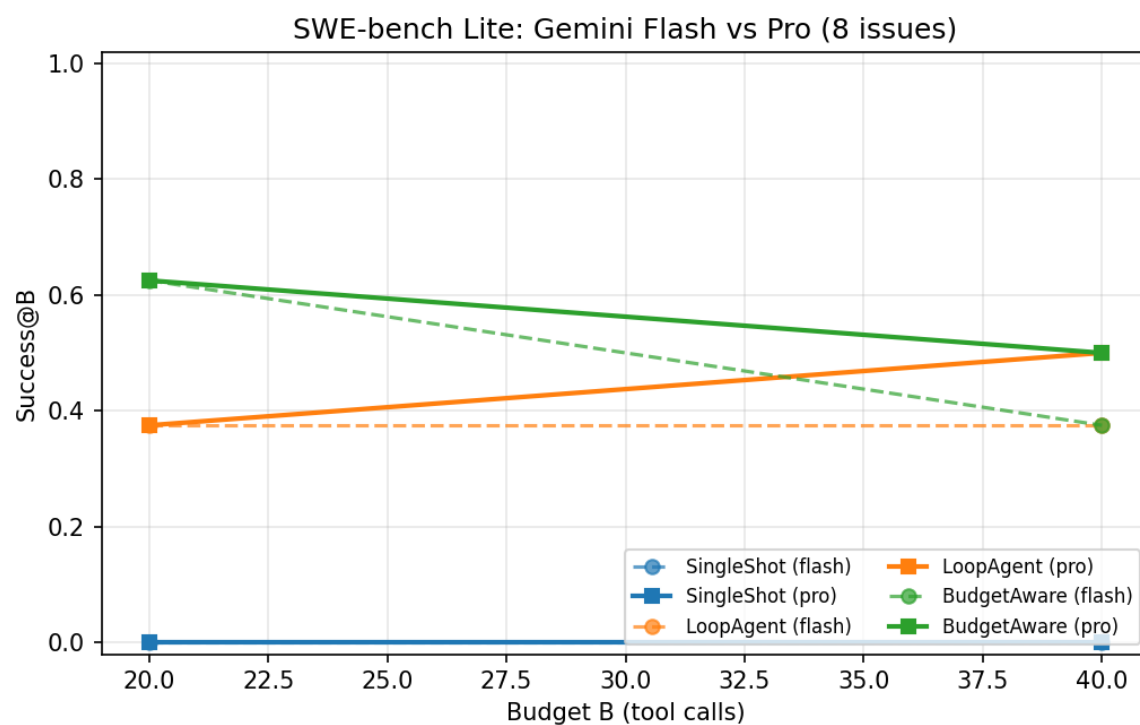


Figure 3: SWE-bench Lite: Success@B for Flash (dashed) vs. Pro (solid). Pro adds zero at $B = 20$ but ~ 12 pp at $B = 40$ for multi-step agents.

at $B = 40$ only one of five is.

4 Discussion

Strategy beats raw capability under budget. Switching from Flash to Pro on SWE-bench at $B = 20$ adds nothing for any agent; at $B = 40$ it adds 12.5 pp for both LoopAgent and BudgetAwareAgent, while preserving the agent ordering. Switching from LoopAgent to BudgetAwareAgent at $B = 10$ on Knapsack adds more than 50 pp, flipping the ordering, and at SWE-bench $B = 20$ adds 25 pp under either model. The cost of adding budget awareness was about 120 lines of Python; even where Pro helps, switching agent strategy helps more.

The harness is cheap to add. The runner, metrics, and visualisation modules together are roughly 700 lines of pure Python with no LLM dependencies. The same runner could be retrofitted to SWE-bench, ToolBench, or any other tool-using benchmark by treating the existing benchmark as the Environment and capping per-episode action count.

Design recommendations. First, prefer an early-commit baseline: SingleShot is hard to beat on AUC because it pays one unit then nothing, so explorers must justify their cost with measurable accuracy gains. Second, expose the remaining budget to the agent and act on it; the single intervention that flipped LoopAgent’s behaviour was telling the agent its remaining budget and giving it a fallback path. Third, treat `run_tests` as the dominant cost item and prune it under tight budgets.

Limitations. Eight SWE-bench issues yield wide CIs (e.g., $[0.25, 0.88]$ for the strongest BudgetAware cell); scaling to the full 300-issue Lite split is the most useful follow-up. The Knapsack “Flash versus Pro” contrast is not meaningful in our setup because the Knapsack agents are deterministic heuristics that do not call an LLM; an early draft of this paper reported an ~ 8 pp Pro lift on Knapsack which we attribute to differences in Python’s ran-

domized string-hash seed between runs and have removed from the headline. BudgetAware’s 0.6 and 0.3 thresholds are hand-tuned and held fixed; learning them per environment is open work. The 0% violation rate of BudgetAware is partly by construction, since the LOW-regime fallback always issues `submit_patch`; LoopAgent’s 38% violation rate at SWE-bench $B=20$ Flash is the comparable real failure mode. Budget here counts tool calls, not tokens or wall-clock. We tested only the Gemini 2.5 family; replicating with Claude or GPT would test universality. Our original proposal included retrieval augmentation, which we descope to keep the budget-awareness comparison clean.

5 Conclusion

Existing code-agent benchmarks score the answer and ignore the cost of producing it. BudgetBench treats the tool-call budget as the central axis, intercepts every action, logs every cost, and reports Success@B, AUC, and violation rate with bootstrap intervals. Across 546 episodes a regime-switching BudgetAwareAgent matched or beat the single-shot baseline with zero violations while a budget-blind ReAct loop collapsed at low budgets. Switching from Gemini Flash to Pro on SWE-bench added nothing at $B = 20$ but 12.5pp at $B = 40$ for multi-step agents, preserving the agent ordering in every cell. The harness is small, LLM-agnostic, and could be retrofitted to existing benchmarks with a thin shim, turning every tool-using-agent paper into one that also reports its action efficiency. Code at <https://github.com/Eric-XC/budgetbench>.

References

- (1) Jimenez, C. E.; Yang, J.; Wettig, A.; Yao, S.; Pei, K.; Press, O.; Narasimhan, K. SWE-bench: Can Language Models Resolve Real-world Github Issues? The Twelfth International Conference on Learning Representations (ICLR). 2024; arXiv:2310.06770.
- (2) Chen, M. et al. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* **2021**,
- (3) Austin, J.; Odena, A.; Nye, M.; Bosma, M.; Michalewski, H.; Dohan, D.; Jiang, E.; Cai, C.; Terry, M.; Le, Q.; Sutton, C. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* **2021**,
- (4) Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; Cao, Y. ReAct: Synergizing Reasoning and Acting in Language Models. The Eleventh International Conference on Learning Representations (ICLR). 2023; arXiv:2210.03629.
- (5) Yang, J.; Jimenez, C. E.; Wettig, A.; Lieret, K.; Yao, S.; Narasimhan, K.; Press, O. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. Advances in Neural Information Processing Systems (NeurIPS). 2024; arXiv:2405.15793.
- (6) Liang, P. et al. Holistic Evaluation of Language Models. *Transactions on Machine Learning Research* **2023**, arXiv:2211.09110.
- (7) Zilberstein, S. Using Anytime Algorithms in Intelligent Systems. *AI Magazine* **1996**, 17, 73–83.
- (8) Yang, J.; Jimenez, C. E.; Press, O.; Narasimhan, K. SWE-bench Lite: A Canonical Subset for Efficient Evaluation of Language Models as Software Engineers. <https://www.swebench.com/lite.html>, 2024; Curated subset of Jimenez et al.¹; 300 issues selected for tractable evaluation.